

Parallel Programming

Converting sequential programs $\rightarrow$ parallel is usually not too hard.

Optimising $\parallel^l$ programs $\rightarrow$ v. hard.

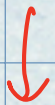
Ex) Recursion $result[i] = a[i] * result[i-1] + result[i-2]$

Technique: prefix sum

Ex) $\vec{d} = \vec{a} * \vec{b} * \vec{c}$

$$\Rightarrow d[i] = a[i] * b[i] * c[i]$$

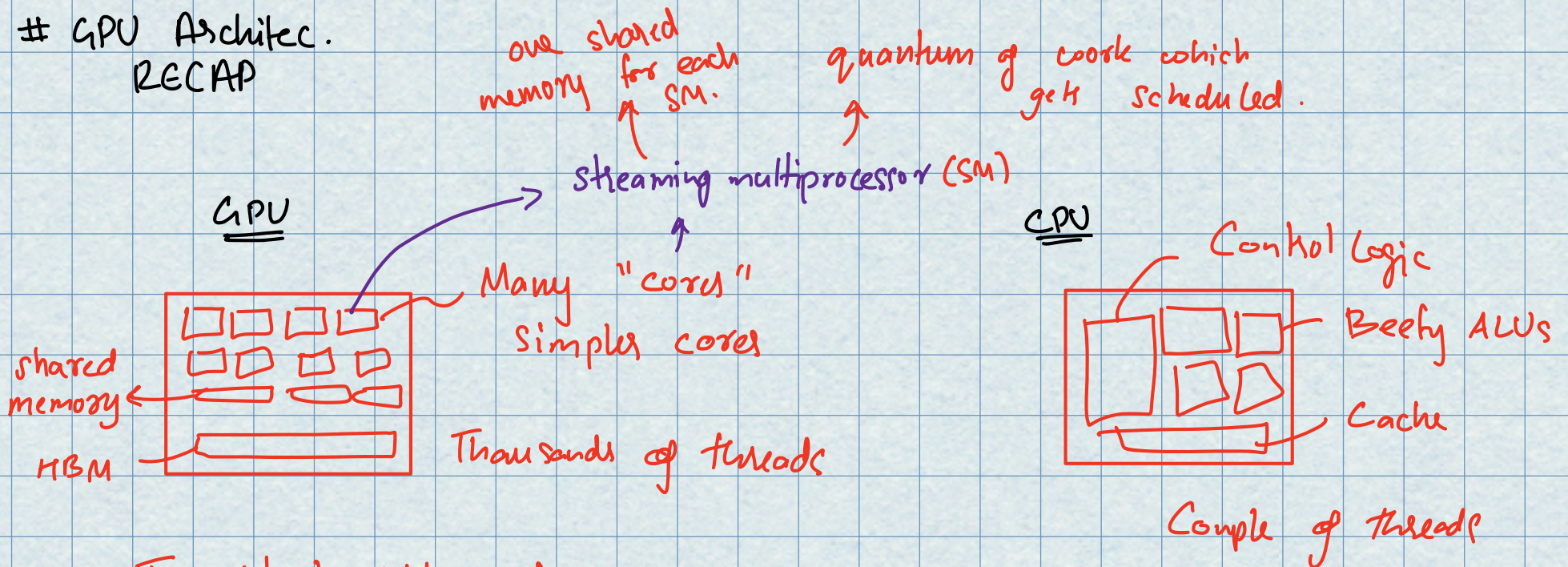
$\hookrightarrow$ embarrassingly $\parallel^l$ program (or) data parallel



When doing something like this, we should be not go back to RAM again & again.

$\hookrightarrow$ efficient memory access
 $\hookrightarrow$ shared memory for all threads to use.

GPU Architec. RECAP



Throughput Optimised

Latency Optimised.

— . — . — . CUDA — . — . — .

PyTorch function call stack

`torch.matmul(A, B)`

↓

· · · pytorch · · · } bunch of internal calls

↓

· · · · ·

↓

`gemm << · · >>` } CUDA

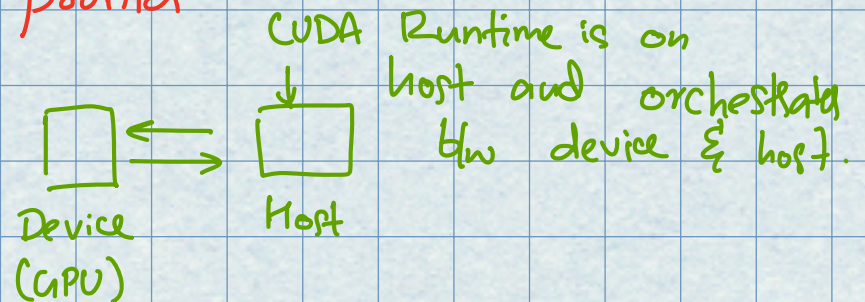
Why learn CUDA

- understand GPU
- write better programs
- implement & improve GPU algorithms for new (DL/AI) architecture.

What is CUDA

Compute Unified Device Arch.

- Extension to C++
- has compiler, debugger, profiler
- Runtime environment



Data Parallelism

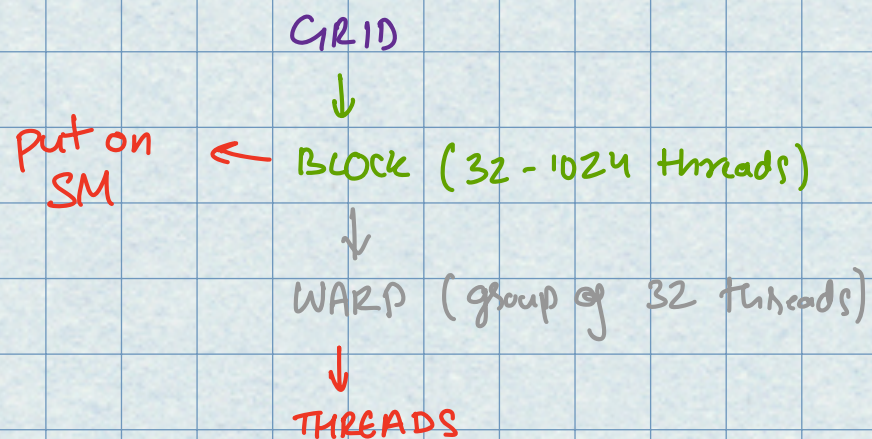
Ex) Vector Addition

$$\begin{array}{c} \vec{a} \\ \vec{b} \end{array} \begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 1 \\ \hline \end{array} \begin{array}{c} + \\ + \\ + \\ + \end{array} \begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 1 \\ \hline \end{array} = \vec{c}$$

Ex) Pixel wise image ops.

- Same '+' operation applied to multiple data.
- programmer needs to exploit

GPU Architecture for CUDA:



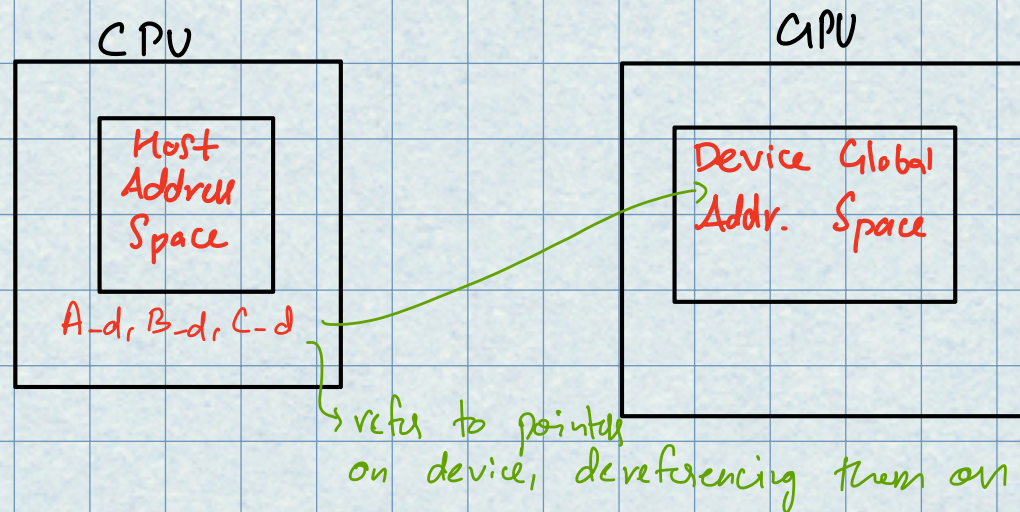
CUDA - Extension of C++ - SPMD: single program multiple data
(not same as single instructⁿ MD)
SIMD

Ex) Vector Addition

CPU :-

```
void add (float * A_h, float * B_h, float * C_h, int n) {  
    for (int i = 0 ; i < n ; i++) {  
        C_h[i] = A_h[i] + B_h[i];  
    }  
}
```

Address Space in CUDA



Convert to GPU: need to move data from host to device

Execution on CPU, GPU is ASYNCHRONOUS.

→ when CPU calls `vecAddKernel`, it doesn't wait
→ it continues on to do things.

LOOK AT THIS FIRST

CPU (host)
runs serially

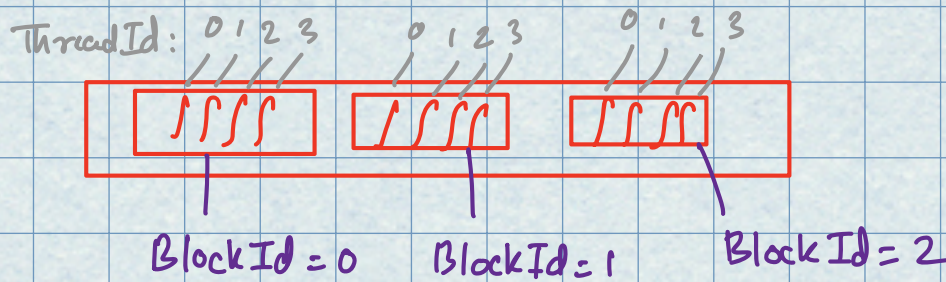
```
void vecAdd(A-h, B-h, C-h, n) {  
    float * A-d, B-d, C-d;  
    // allocate on device // copy data  
    int blockSize = 256;  
    int numBlocks = ceil(n/blockSize);  
    vecAddKernel <<< numBlocks, blockSize >>> (A-d,  
                                                B-d, C-d, n);  
}
```

Fill in later.

GPU (device) - CUDA kernel

```
-- global -- void vecAddKernel(float * A-d,  
                                float * B-d, float * C-d, int n) {  
    int i = threadIdx.x + blockDim.x *  
            blockIdx.x  
    if (i < n)  
        C-d[i] = A-d[i] + B-d[i];  
}
```

Consider



$$\text{int } i = \underbrace{\text{threadId} \cdot X}_{\text{Thread ID}} + \underbrace{\text{blockDim} \cdot X}_{\text{no. of threads in a block}} * \underbrace{\text{blockId} \cdot X}_{\text{Block ID}}$$

global index of a thread.

• X is because blocks can be 3 dimensional
ex) when data is 2D or 3D.

After kernel executed, get data back to host and free memory.

CPU (host)

```
void vecAdd(A-h, B-h, C-h, n) {
```

```
    float * A-d, B-d, C-d;
```

```
    // allocate on device // copy data
```

```
    vecAddKernel <<< numBlocks, blockSize >>> (A-d, B-d, C-d, n);
```

```
    cudaDeviceSynchronize();
```

```
    // copy device → host // free device memory
```

```
}
```

To wait for GPU as in this case we don't have other thing to do for CPU.

// Allocate on Device

```
int size = n * sizeof(float);  
cudaMalloc((void**) &A-d, size);  
"          &B-d    "  
"          &C-d    "
```

& to pass reference
so cudaMalloc can
change value.

void** is how CUDA
wants those pointers
typed as.

// Copy host to device

```
cudaMemcpy(A-d, A-h, size, cudaMemcpyHostToDevice);  
          dst src  
"      B-d, B-h  "
```

// Copy device to host

```
cudaMemcpy(C-h, C-d, size, cudaMemcpyDeviceToHost);
```

// Free

```
cudaFree(A-d);  
"      B-d  "  
"      C-d  "
```

#

→ can only return void.

--global --void vccAddKernel . . .

↳

Execution Specifier

↳ for compiler to be able compile & link w/ host C++ code.

--host -- (default)

executes on host

↓

--device --

↳ won't encounter usually, for code running on GPU.

Execution Configuration : execution hyperparams.

↑

vccAddKernel <<< numBlocks, blockSize >>> (A-d, B-d, C-d, n)

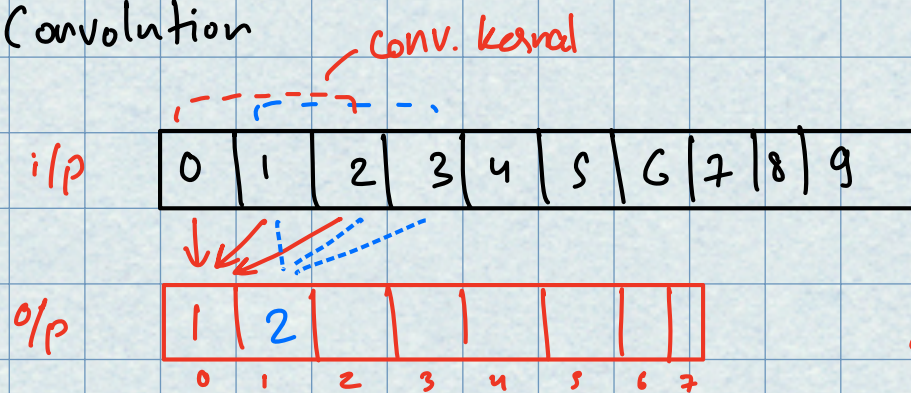
inside CHEVRON symbols.

CUDA compiler : NVCC

→ invoked like C-compiler

→ returns executable which we run.

ex) Convolution

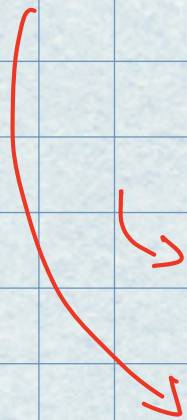


$$o/p[i] = (i/p[i] + i/p[i+1] + i/p[i+2]) / 3.0;$$

```
-- global -- void convolve (float * input , float * output , int N) {  
    int idx = threadIdx.x + blockDim.x * blockIdx.x ;  
    int res = 0 ;  
    if (idx < N) {  
        for (int j=0 ; j < 3 ; j++)  
            res += input [idx+j] ;  
        output[idx] = res ;  
    }  
}
```

memory reads are expensive.

here, each element in i/p is accessed in general 3 times and loaded in each thread multiple times.

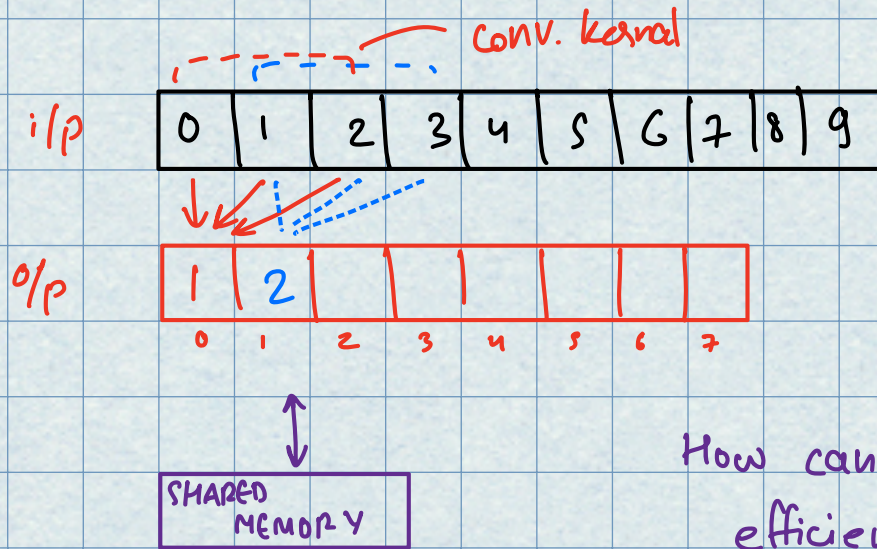


can use shared memory.

now : $\sim 8 \times 3 = 24$ reads

ideally : 10 reads

Ex) Convolution w/ shared memory



$$o/p[i] = (i/p[i] + i/p[i+1] + i/p[i+2]) / 3.0;$$

How can we cooperate b/w threads for efficient memory usage through shared memory?

in CUDA, code is written s.t. every thread will do that.

This keyword means only once this will happen

```
#define THREADS_PER_BLOCK 256
```

```
--global-- void convolve(float *inp, float *out, int N) {
```

```
    --shared-- float buffer[THREADS_PER_BLOCK + 2];
    int idx = threadIdx.x + blockDim.x * blockIdx.x;
```

each thread is one o/p, kernel size is 3, 2 remaining no.s to account for.

```
    buffer[threadIdx.x] = inp[idx];
```

Loading data in shared buffer.

if (ThreadId.x < 2)

buffer[THREADS_PER_BLOCK + ThreadId.x]

= inp[THREADS_PER_BLOCK + idx];

-- syncThreads() --;

result = 0.0f

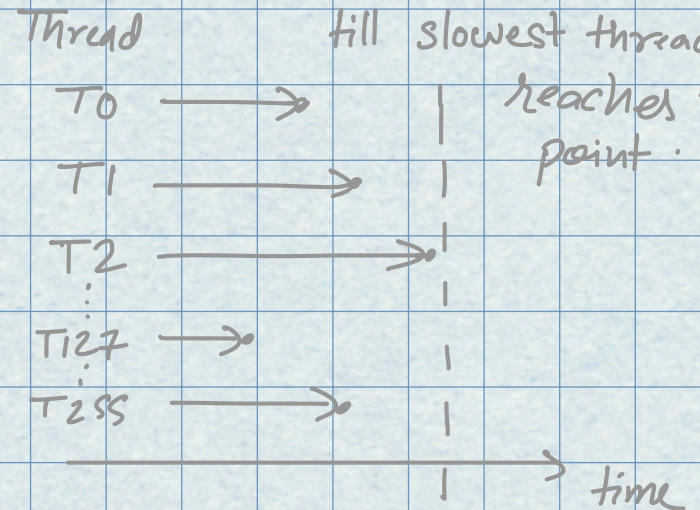
for (int i = 0; i < 3; i++)

result += buffer[ThreadId.x + i];

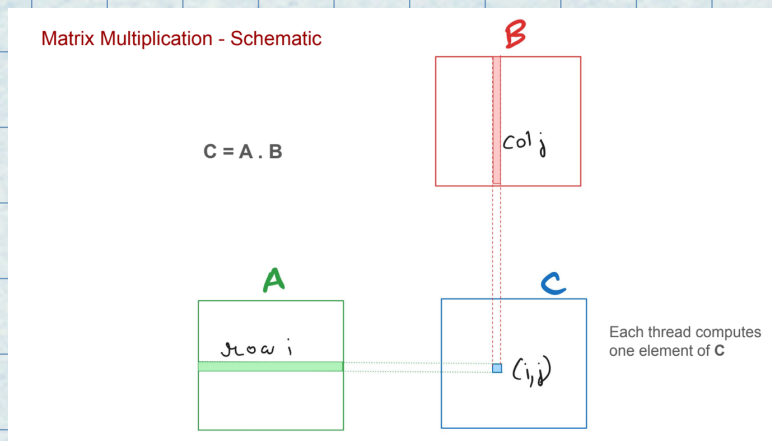
out[idx] = result / 3.0f;

core
logic

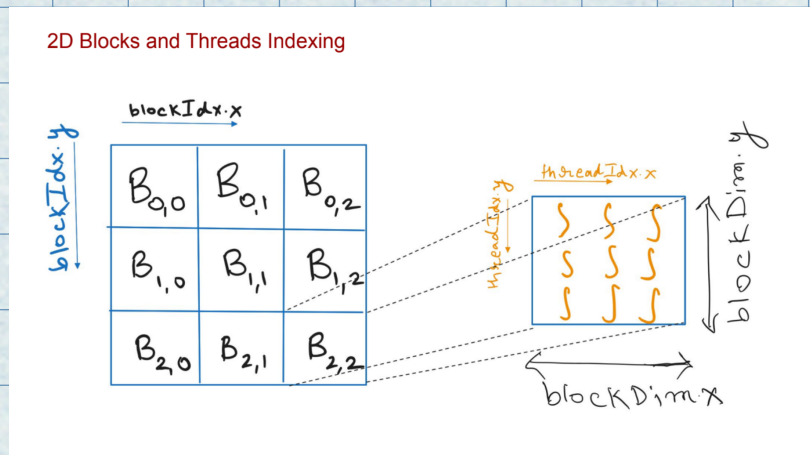
all threads wait
till slowest thread
reaches this
point.



MATRIX MUL Ex.



Each thread can take care of one o/p element, which is a row and col dot product.



Data is 2D, good to have GRID defined in 2D.

BTW: these structs, blockIdx, threadIdx, are kept in registers by CUDA runtime.

Looked at simple mat mul code:

I/P Data: $N \times N$ matrix

no. of threads per block : 128 - 1024
(one warp per subcon) (env. limit)

256
is good usually.

256 threads in 2D block
 $\Rightarrow$ block size = 16×16

no. of blocks : each block does 16×16 entries
256 o/p's

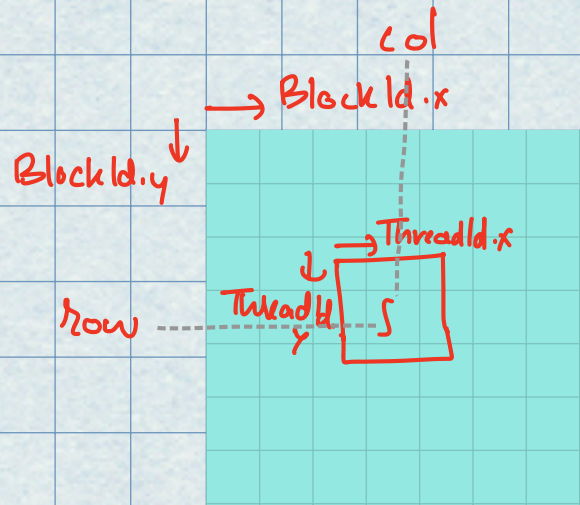
$N \times N$ elements

$$\text{no. of blocks} = \frac{N \times N}{256}$$

$$\text{no. of rows of blocks or cols} = \frac{N}{16}$$

we take ceil of this $\nearrow$

How will a thread know which element to compute?



$$\begin{aligned} \text{row} &= \text{blockId.x.y} * \text{blockDim.y} + \text{ThreadId.y} \\ \text{col} &= \text{blockId.x.y} * \text{blockDim.x} + \text{ThreadId.x} \end{aligned}$$

inp: $x[N \times N]$, $y[N \times N]$ in row major format

if we don't have { if idle threads don't run
this, these out of bound threads will
still access out of
bounds data and
mess up your o/p.

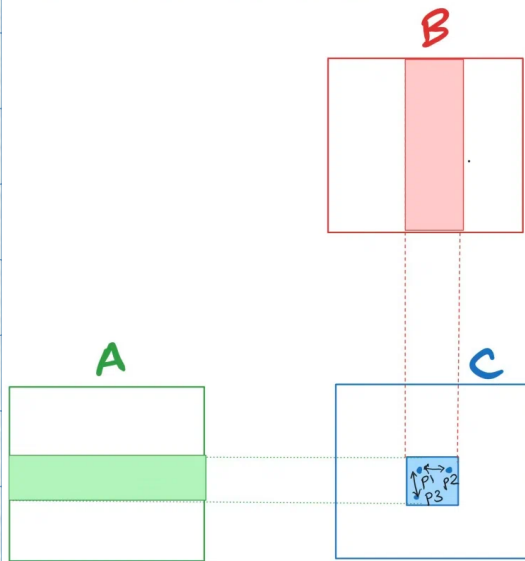
dot
product

```
if (row < N && col < N)
{
    float out_val = 0.0f;
    for (int m = 0; m < N; m++)
    {
        out_val += (x[row * N + m] * y[m * N + col]);
    }
    out[row * N + col] = out_val;
}
```

clever indexing

Tiled MatMul Ex

Towards Tiled Matrix Multiplication



- Recall reads from global HBM is slow
- p1 and p2 are working with same row x of A
- p1 and p3 are working with same col y of B

$$\begin{aligned} p1 &= (x, y) \\ p2 &= (x, a) \\ p3 &= (b, y) \end{aligned}$$

p_1 and p_2 share some data needed to compute.
(same row from A)

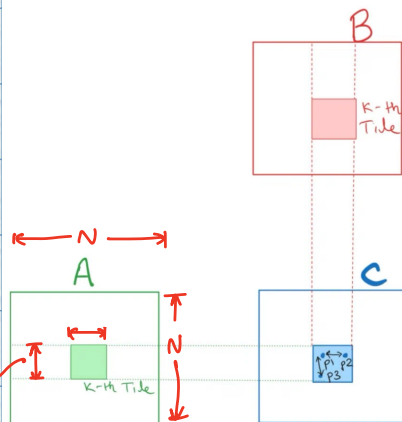
We can make use of shared memory.

on AI00:

65.5 K reg of 32 bit
 $\therefore$ 256 KB of registers

and 128 KB of shared memory.

Tiled Matrix Multiplication Implementation



For each tile of output C:

- Iterate over tiles in A and B
- Tiles of A and B are likely small $\rightarrow$ will fit in small SM shared memory

$$\begin{aligned} p1 &= (x, y) \\ p2 &= (x, a) \\ p3 &= (b, y) \end{aligned}$$

We can perform the multiplications in a tiled fashion w/ shared memory & accumulate results as we go.

$$\# \text{tiles} = \frac{N}{\text{block-size}}$$

```
#define BLOCK_SIZE 16
```

```
#define THREADS_IN_BLOCK = BLOCK_SIZE * BLOCK_SIZE // 256 threads
```

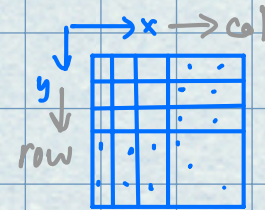
```
__global__ void matMulShared(float* a, float* b, float* c, int N) { // a, b, c in row major
```

```
    int row = threadIdx.y + BlockDim.y * blockIdx.y;
```

```
    int col = threadIdx.x + blockIdx.x * blockDim.x;
```

```
    int tx = threadIdx.x;
```

```
    int ty = threadIdx.y;
```



runs once per block {

```
    __shared__ float a-tile[BLOCK_SIZE][BLOCK_SIZE];  
    __shared__ float b-tile[BLOCK_SIZE][BLOCK_SIZE];
```

```
    float out = 0.0f; // accumulator
```

```
    for(k=0; k < N/BLOCK_SIZE; k++) {
```

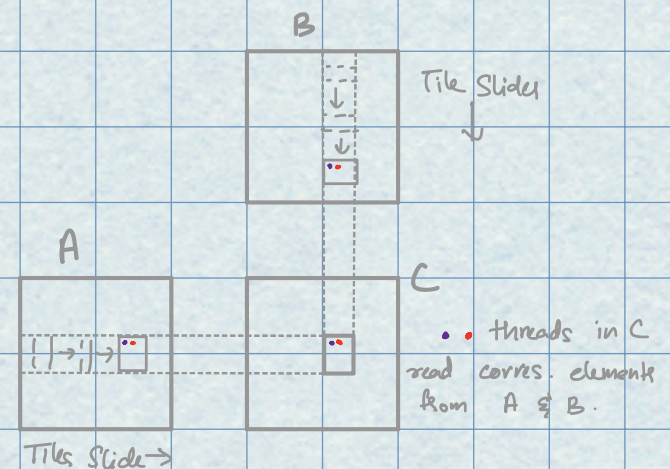
```
        int offset = k * BLOCK_SIZE;
```

```
        a-tile[tx][ty] = a[row*N + offset + tx];
```

```
        b-tile[tx][ty] = b[(offset + ty)*N + col];
```

```
        __syncthreads();
```

} so a different /



```

for (m=0; m < BLOCK_SIZE; m++)
    out += (a_tile[ty][m] * b_tile[m][ty]);
    -- syncThreads();
}

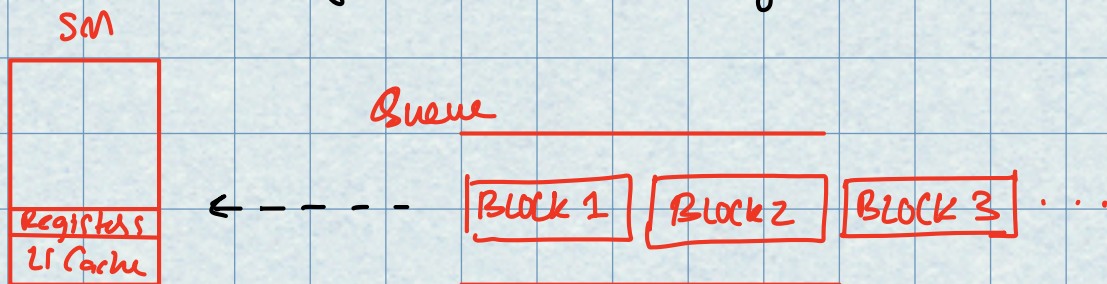
```

```

c[row * N + col] = out;
}

```

Q. Is there enough shared memory?



no. of registers needed / thread depends on your code.

if your block needs many registers, scheduler will only run one or two blocks.

so the limiting resource (#registers or shared mem.) decides how many blocks get scheduled.

ex) Coordination b/w Threads

Tensor Core - Does MMA in one instruction for sizes of 16×16
matrix mul. accumulate wmma
warp

$$\left\{ \begin{bmatrix} \quad \end{bmatrix}_{16 \times 16} \times \begin{bmatrix} \quad \end{bmatrix}_{16 \times 16} + \begin{bmatrix} \quad \end{bmatrix}_{16 \times 16} = \begin{bmatrix} \quad \end{bmatrix}_{16 \times 16} \right\}$$

one warp does this.

instead of thinking per thread task, we now start thinking about tiling & coordination.
At Block or warp level.